

AT Work 镜像 v1.0.0 版本安装部署手册

基于系统镜像为 Rocky Linux 8.4 作部署（停止更新维护，可选择平替系统镜像版本，比如 centos stream8）

一、基础系统初始化

1.1 系统更新

Plain Text

```
sudo yum update -y
```

1.2 安装基础工具包

Plain Text

```
sudo yum install -y yum-utils
```

1.3 手动配置系统时间

Plain Text

```
sudo timedatectl set-ntp true
sudo timedatectl set-timezone Asia/Shanghai
```

二、Docker 安装部署

2.1 卸载旧版本 Docker

Plain Text

```
sudo yum remove docker \
    docker-client \
    docker-client-latest \
    docker-common \
    docker-latest \
```

```
docker-latest-logrotate \  
docker-logrotate \  
docker-engine
```

2.2 安装依赖工具

Plain Text

```
sudo yum install -y yum-utils device-mapper-persistent-data lvm2
```

2.3 添加 Docker 官方仓库

Plain Text

```
sudo yum-config-manager --add-repo  
https://download.docker.com/linux/centos/docker-ce.repo  
# 更新仓库缓存  
sudo yum clean all  
sudo yum makecache  
yum clean packages
```

2.4 更新仓库，安装 Docker

Plain Text

```
sudo yum install -y docker-ce docker-ce-cli containerd.io
```

备注：可能由于网络原因会失败，多执行几次即可

2.5 启动 Docker 并设置开机自启

Plain Text

```
sudo systemctl start docker  
sudo systemctl enable docker
```

三、K3s 集群部署（离线安装）

说明：k3s 采用离线安装模式，相关附属镜像为在线安装

3.1 前置配置（关闭防火墙）

Plain Text

```
# 更新仓库缓存
```

```
systemctl stop firewalld
```

```
# 开机不自启
```

```
systemctl disable firewalld
```

3.2 主节点安装部署

1、将 k3s 文件和 k3s-install.sh 上传至 /usr/local/bin 目录

2、赋予文件可执行权限

Plain Text

```
chmod +x k3s-install.sh
```

```
chmod +x k3s
```

3、配置离线安装环境变量并执行安装脚本

Plain Text

```
export INSTALL_K3S_SKIP_DOWNLOAD=true
```

```
export INSTALL_K3S_BIN_DIR=/usr/local/bin
```

```
export INSTALL_K3S_EXEC="--docker --write-kubeconfig  
~/.kube/config --write-kubeconfig-mode 666"
```

```
# 执行安装脚本
```

```
./k3s-install.sh
```

4、修改 K3s 配置文件

Plain Text

```
vi /etc/rancher/k3s/k3s.yaml
```

将配置中的地址修改为集群本机内网 ip: 172.31.0.2


```
~/ .kube/config --write-kubeconfig-mode 666"
```

4、把主节点获取到的 token 和内网 ip 替换 然后执行下面命令

主节点查询 Token 命令：

Plain Text

```
cat /var/lib/rancher/k3s/server/node-token
```

```
[root@ecs-9d89-0001 bin]# cat /var/lib/rancher/k3s/server/node-token
K1086b8af6b259d5acf8b2fb7ba947ab46a69e3e45f3ac5a7feef9d1e2a12239faa::server:37117a44d5874a30dbe852f528f2d97f
[root@ecs-9d89-0001 bin]#
```

工作节点执行加入集群命令（替换真实 Token 和主节点 IP）：

Plain Text

```
K3S_TOKEN="K107c62c70b37378641bbf2c5cee0f110219c2da53b55165c8a755f
bdcdc408bd7f::server:c7445ddeb0448603e4a83de2087a2bd1"
K3S_URL=https://192.168.3.160:6443 ./k3s-install.sh
```

5、验证集群节点状态

Plain Text

```
kubectl get node -o wide
```

```
Every 2.0s: kubectl get node -o wide                                ecs-9d89-0001: Tue Mar 11 14:18:30 2025
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE	KERNEL-VE
ecs-9d89-0001	Ready	control-plane,master	12m	v1.29.0+k3s1	192.168.0.109	<none>	CentOS Stream 8	4.18.0-55
3.6.1.e18.x86_64		docker://26.1.3						
ecs-9d89-0002	Ready	<none>	18s	v1.29.0+k3s1	192.168.0.92	<none>	CentOS Stream 8	4.18.0-55
3.6.1.e18.x86_64		docker://26.1.3						

3.4 K3s 常用运维命令

Plain Text

```
systemctl status k3s      # 查看服务状态
systemctl stop k3s        # 停止服务
systemctl start k3s       # 启动服务
systemctl restart k3s     # 重启服务
k3s-agent-uninstall.sh    # 卸载 agent 节点
```

四、Rancher 部署与集群导入

4.1 Rancher 容器部署

外网通畅环境执行在线部署命令，直接用 docker 命令运行的 rancher2.5.5 然后用导入集群：

Plain Text

```
sudo docker run -d \  
  --name rancher \  
  --privileged \  
  --restart=always \  
  -p 8088:80 \  
  -p 8443:443 \  
  -v /opt/rancher:/var/lib/rancher \  
  rancher/rancher:v2.5.5
```

外网不通环境就把文件夹里面的 rancher2.5.5-image 镜像解压：

Plain Text

```
docker load -i rancher.tar
```

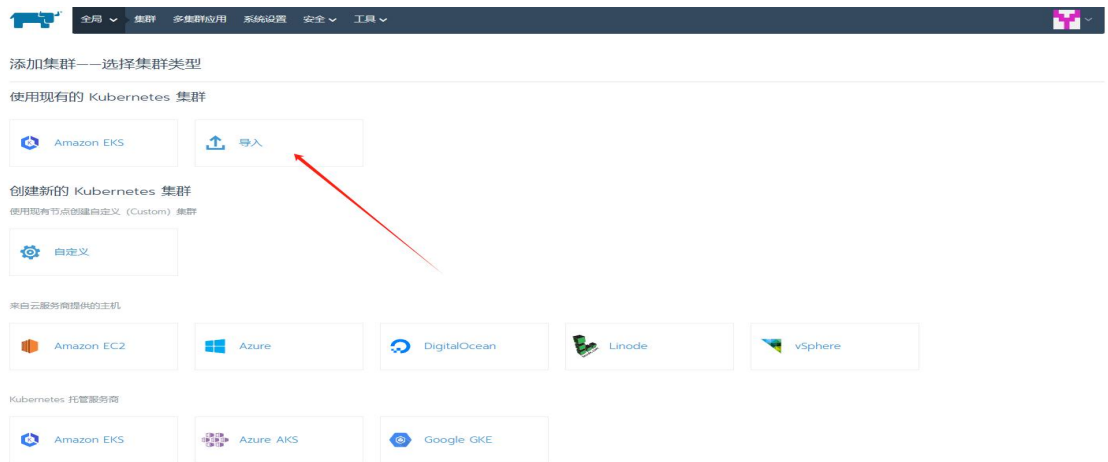
端口放行要求：云平台的控制台需要打开 8088 端口和 8443 端口，运行成功之后 来到先设置登陆密码（非常简单） 进入主界面

4.2 集群导入配置

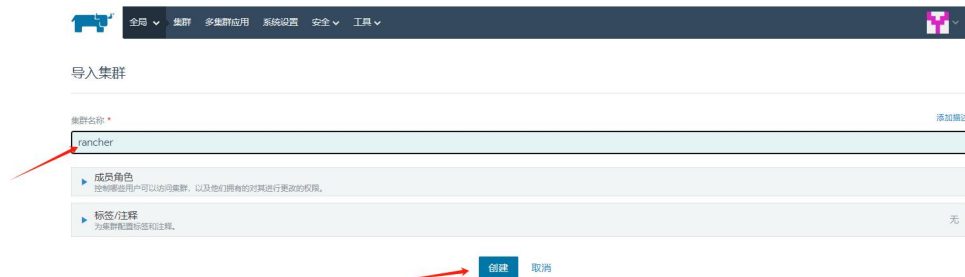
1、启动 Rancher 后初始化登录密码，进入管理主界面



2、忽略默认集群，点击【添加集群】-【导入集群】



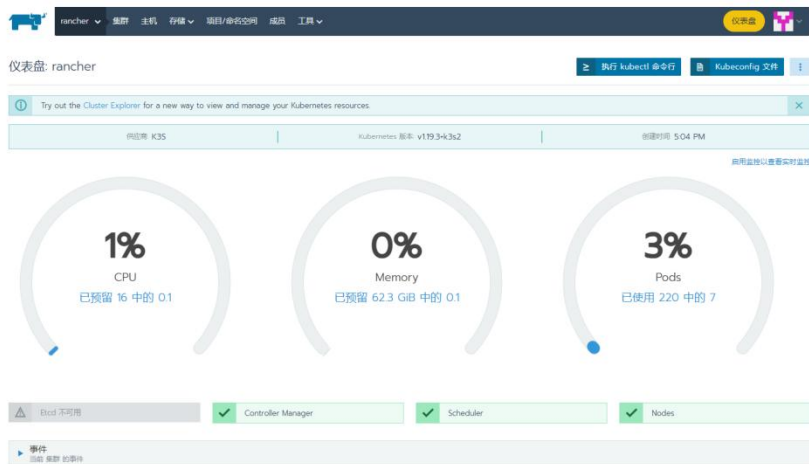
3、自定义集群名称并创建



4、由于这个是直接使用 docker 运行的 rancher 所以选择下面这个命令，到主节点的机器终端直接运行，完事之后等自动构建



5、构建完成之后的主界面和主机页面如下



6、点击主机 可以看到两台机器组成的小集群就已经完成，然后就可以正常的部署了

五、数据盘挂载与 NFS 服务部署

5.1 服务器数据盘挂载

如果是服务器的时候需要优先挂载数据盘，在准备安装 nfs 服务端的服务器上购买一个数据盘

Plain Text

```
lsblk          # 查看磁盘设备
mkfs.ext4 /dev/vdb  # 格式化数据盘
mount /dev/vdb /data  # 挂载磁盘至/data 目录
```

5.2 NFS 服务端部署配置

NFS 作用：实现集群节点间文件资源共享，保障 Pod 跨节点访问文件

1、安装 NFS 服务（NFS 是一套网络文件系统，允许计算机之间共享资源）

Plain Text

```
yum install nfs-utils -y
```

2、创建共享目录

```
Plain Text
mkdir -p /mnt-at
mkdir -p /data
```

3、配置共享规则（配置可访问的客户端 ip 地址，需要共享哪些文件就配置； rw 表示读写权限，sync 表示同步写入）

在 `etc/exports` 文件中加入配置

```
Plain Text
/data/nfs-share *(insecure,rw,sync,no_root_squash)
/data/redis *(insecure,rw,sync,no_root_squash)
/data/mysql/k8s *(insecure,rw,sync,no_root_squash)
/data/documentserver *(insecure,rw,sync,no_root_squash)
/data/ide-share *(insecure,rw,sync,no_root_squash)
/mnt-at *(insecure,rw,sync,no_root_squash)
/data/anythingllm *(insecure,rw,sync,no_root_squash)
/data/wiki *(insecure,rw,sync,no_root_squash)
```

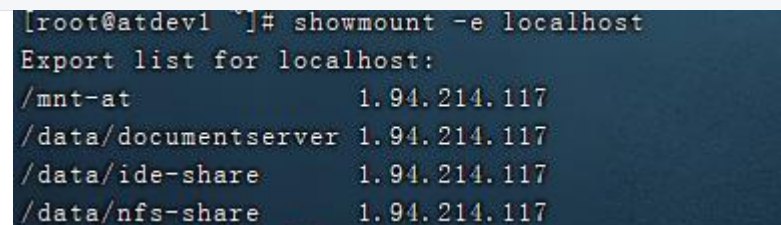
4、生效配置并启动服务

```
Plain Text
exportfs -ra

# 开 启 并 设 置 开 机 自 启
systemctl start nfs-server
systemctl enable nfs-server
```

5、验证共享配置

```
Plain Text
showmount -e localhost
```



```
[root@atdev1 ~]# showmount -e localhost
Export list for localhost:
/mnt-at 1.94.214.117
/data/documentserver 1.94.214.117
/data/ide-share 1.94.214.117
/data/nfs-share 1.94.214.117
```

5.3 NFS 客户端配置（所有集群节点）

1、安装客户端工具

```
Plain Text
yum install nfs-utils
```

2、编辑开机自动挂载文件

```
Plain Text
#          修          改          配          置          文          件
/etc/fstab
```

#服务端 ip 以及共享的目录

```
Plain Text
192.168.3.152:/mnt-at /mnt-at  nfs  defaults 0 0
192.168.3.152:/data/documentserver /data/documentserver  nfs
defaults 0 0
192.168.3.152:/data/mysql/k8s /data/mysql/k8s  nfs  defaults 0 0
192.168.3.152:/data/nfs-share /data/nfs-share  nfs  defaults 0 0
192.168.3.152:/data/redis /data/redis  nfs  defaults 0 0
192.168.3.152:/data/ide-share /data/ide-share  nfs  defaults 0 0
192.168.3.152:/data/anythingllm /data/anythingllm  nfs  defaults 0
0
192.168.3.152:/data/wiki /data/wiki  nfs  defaults 0 0
```

3、#挂载 NFS 共享

```
Plain Text
systemctl daemon-reload
mount -a
```

4、验证挂载结果

```
Plain Text
df -h
```

六、AT Work 服务部署

6.1 前置准备

1、创建 Nacos 命名空间

```
Plain Text
kubectl create namespace nacos
```

2、将 `nacos-k8s-master` 和 `nacos` 目录上传至服务器 `/opt` 目录

6.2 部署 NFS 存储类

进入 `/opt/nacos` 目录执行以下部署命令，修改配置文件内服务端 IP，保证 `PROVISIONER_NAME` 参数与集群存储类名称一致

```
Plain Text
kubectl apply -f nfs-client-rbac.yaml
kubectl apply -f nfs-client-provisioner.yaml
# 查看集群存储类名称
kubectl get storageclass
```

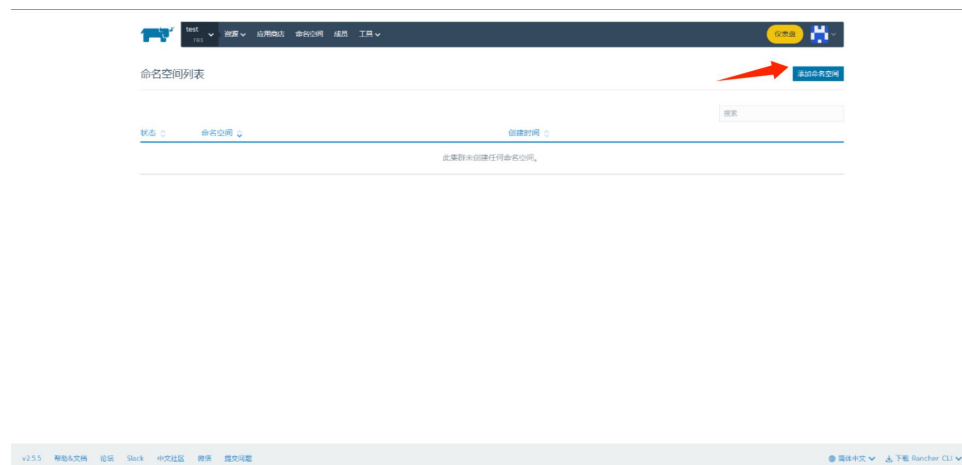
注意：配置 `PROVISIONER_NAME` 参数的时候的值一定要配置正确，可以按照以下命令：
`kubectl get storageclass`

```
[root@atdev001 mysql]# kubectl get storageclass
NAME                                PROVISIONER                                RECLAIMPOLICY    VOLUMEBINDINGMODE    ALLOWVOLUMEEXPANSION    AGE
local-path (default)               rancher.io/local-path                     Delete           WaitForFirstConsumer  false                   2d
managed-nfs-storage                 fuseim.pri/ifs                             Delete           Immediate              false                   3h2m
```

一定要与途中标注的名字保持一致

6.3 MySQL 部署

1、Rancher 端创建项目 `res`（用于部署在 k3s 运行的中间件服务），可同步创建命名空间或者在主节点上用命令创建



2、部署 `mysql`（这个 `mysql` 用于生产数据库和 `nacos` 配置的数据库），前提是得把上面的 `nfs` 环境安装配置好，不然部署会报错。

进入到 `/opt/nacos-k8s-master/deploy/mysql`

`kubectl apply -f mysql-nfs.yaml`

运行成功之后，去 `rancher` 的页面是这样的



3、初始化 `Nacos` 数据库账号

连接 `MySQL` 数据库，执行以下 `SQL` 添加登录用户：

Plain Text

```
INSERT INTO `nacos`.`users` (`username`, `password`, `enabled`)
VALUES ('nacos',
'$2a$10$1I08WgYcN57UcHihW00ucezt6mAuLe1GkJJWlAwafRwRR/HMr3pKy',
1);
```

6.4 Nacos 部署

1、如果没有网络或者不能顺利拉下镜像的情况下就解压一下 `/opt/nacos-k8s-master/deploy/nacos` 的两个镜像

```
docker load -i nacos-sever.tar nacos-peer.tar
```

nacos-pvc-nfs.yaml 文件（目前存放在 /opt/nacos-k8s-master/deploy/nacos 目录下）进入该目录下直接运行以下命令

```
cd /opt/nacos-k8s-master/deploy/nacos
kubectl apply -f nacos-pvc-nfs.yaml
```

Active
nacos
80/http
nacos/nacos-server:2.0.3 · 其他 1 个 image
3 个 Pods · 创建时间: 14 hours ago · 容器重启次数: 0

编辑规则

名称
nacos

添加描述
中心空间
nacos

规则

☐ 继续使用当前主机名

☐ 自动选择 `server` 云环境域名

☒ 自定义域名

☐ 默认后端

主机名

nacos.alibaba.com.cn

① 如果目标后端是一个服务，您可以使用该服务所暴露的端口。您可以在服务发现页找到该服务，然后通过编辑 YAML 的方式添加所需要的端口。

目标后端

+

服务

+

工作负载

访问路径(如使用目标后端的鉴权功能, 请查看下方标题/注释):
例如 /foo

服务/工作负载
nacos-headless

端口 *
8548

全部需

添加规则

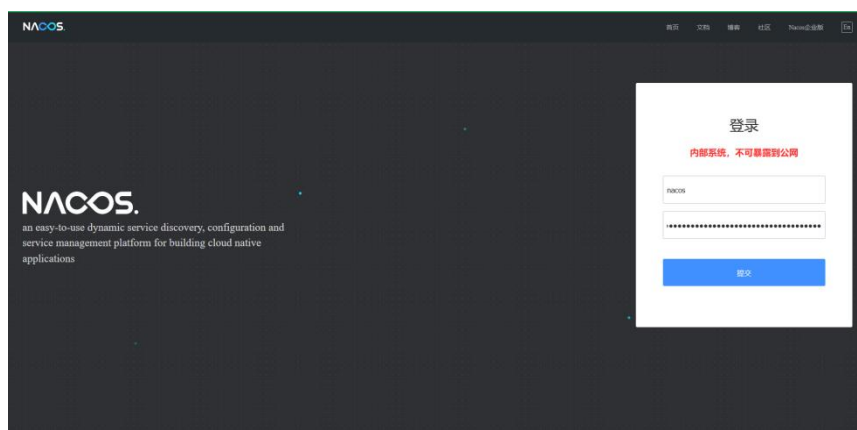
SSL/TLS 证书
配置用于端口加密的 SSL/TLS 证书。

配置/设置

```
192.168.3.150 nacos-opensource.allthinker.com
```

账号: `nacos`

密码: hufuWkkB8npmXMjrmJqCLtUB123



6.5 XXL-JOB 定时任务部署

1、基础配置

Rancher 创建命名空间 `job`, 镜像使用 `xuxueli/xxl-job-admin:2.2.0`, 离线环境导入镜像:

```
Plain Text
docker load -i xxl.tar
```

在 rancher 上修改修改镜像拉取策略为: 不存在则拉取

2、添加环境变量

键: `PARAMS`

值:

```
Plain Text
--
spring.datasource.url=jdbc:mysql://mysql.nacos:3306/xxl_job?useUnicode=true&characterEncoding=UTF-8&autoReconnect=true&serverTimezone=Asia/Shanghai --
spring.datasource.username=xxl --
spring.datasource.password=sdywtvtEqFvkJHbG3RQDUjqN
```

配置容器可访问的环境变量，包括从其他容器或主机环境变量注入数据。

主机调度

配置 Pod 对应的主机调度规则。

健康检查

周期性向容器发出请求，以检测其健康状态。默认设置下，readiness 和 liveness 使用相同的配置参数。对于应用初始化较长的容器，需要增加就绪检测时间。

数据卷

持久化并共享独立于单个容器生命周期数据的数据。

缩放/升级策略

配置升级过程中替换 Pod 的策略。

入口/命令

配置容器启动时执行的可行执行文件。

网络

设置容器的网络和 DNS 选项。

标签/注释

用于描述对象的元数据，应设置与配置主键 键 nginx.ingress.kubernetes.io/rewrite-target, 值 /。

安全/主机设置

授予或限制容器访问所运行主机的能力。

镜像拉取策略

特权模式

提升特权

☐ 总是拉取

☒ 不存在则拉取

☐ 从不

☒ 否

☐ 是

☒ 否

☐ 是 容器具有所有主机访问权限

☒ 否

☐ 是 容器可以获得比其父进程更多的特权

使用主机的 PID 命名空间

使用主机的 IPC 命名空间

☒ 否

☐ 是

☒ 否

☐ 是

配置容器可访问的环境变量，包括从其他容器或主机环境变量注入数据。

主机调度

配置 Pod 对应的主机调度规则。

健康检查

周期性向容器发出请求，以检测其健康状态。默认设置下，readiness 和 liveness 使用相同的配置参数。对于应用初始化较长的容器，需要增加就绪检测时间。

数据卷

持久化并共享独立于单个容器生命周期数据的数据。

缩放/升级策略

配置升级过程中替换 Pod 的策略。

入口/命令

配置容器启动时执行的可行执行文件。

网络

设置容器的网络和 DNS 选项。

标签/注释

用于描述对象的元数据，应设置与配置主键 键 nginx.ingress.kubernetes.io/rewrite-target, 值 /。

安全/主机设置

授予或限制容器访问所运行主机的能力。

镜像拉取策略

特权模式

提升特权

☐ 总是拉取

☒ 不存在则拉取

☐ 从不

☒ 否

☐ 是

☒ 否

☐ 是 容器具有所有主机访问权限

☒ 否

☐ 是 容器可以获得比其父进程更多的特权

使用主机的 PID 命名空间

使用主机的 IPC 命名空间

☒ 否

☐ 是

☒ 否

☐ 是

3、数据库初始化

配置 xxljob （主要是配置 mysql 的链接 在 mysqlpod 中进入容器输入以下命令）

-- 创建数据库和用户并授权进入容器内部之后先登录 MySQL 数据库（mysql -u root -p 密码：gbQjQow8W\$683zJJNTw&=aqT）， 登陆后执行初始化命令：

Plain Text

```
CREATE DATABASE xxl_job;  
CREATE USER 'xxl'@'%' IDENTIFIED BY 'sdywtvtEqFvkJHbG3RQDUjqN';  
GRANT ALL PRIVILEGES ON xxl_job.* TO 'xxl'@'%';  
FLUSH PRIVILEGES;
```

```

bash-4.2# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 1498
Server version: 5.7.40 MySQL Community Server (GPL)

Copyright (c) 2000, 2022, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
mysql> CREATE USER 'xxl'@'%' IDENTIFIED BY 'sdywtvtEqFvkJHbG3RQDUjqN';
Query OK, 0 rows affected (0.00 sec)

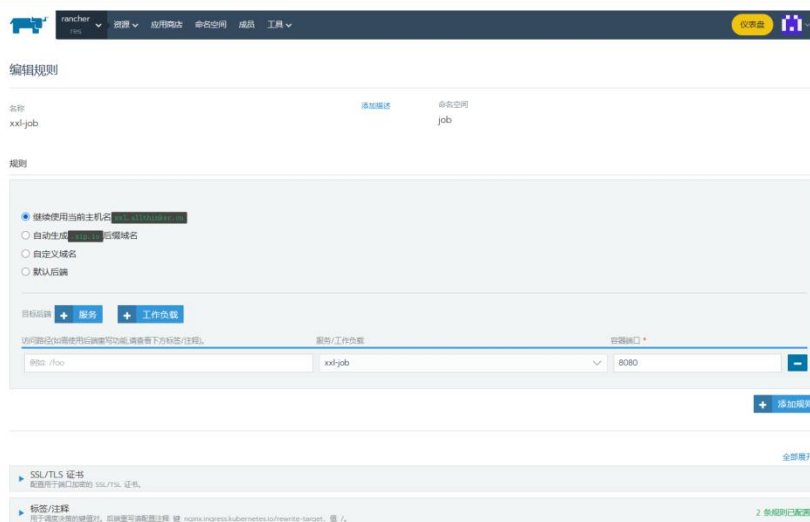
mysql> GRANT ALL PRIVILEGES ON xxl_job.* TO 'xxl'@'%';
Query OK, 0 rows affected (0.00 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.01 sec)

mysql>

```

执行 `/res/xxl-job/xxl_job.sql` 脚本初始化数据表，配置 xxl-job 负载均衡



4、域名访问配置

本机 hosts 添加解析：

```

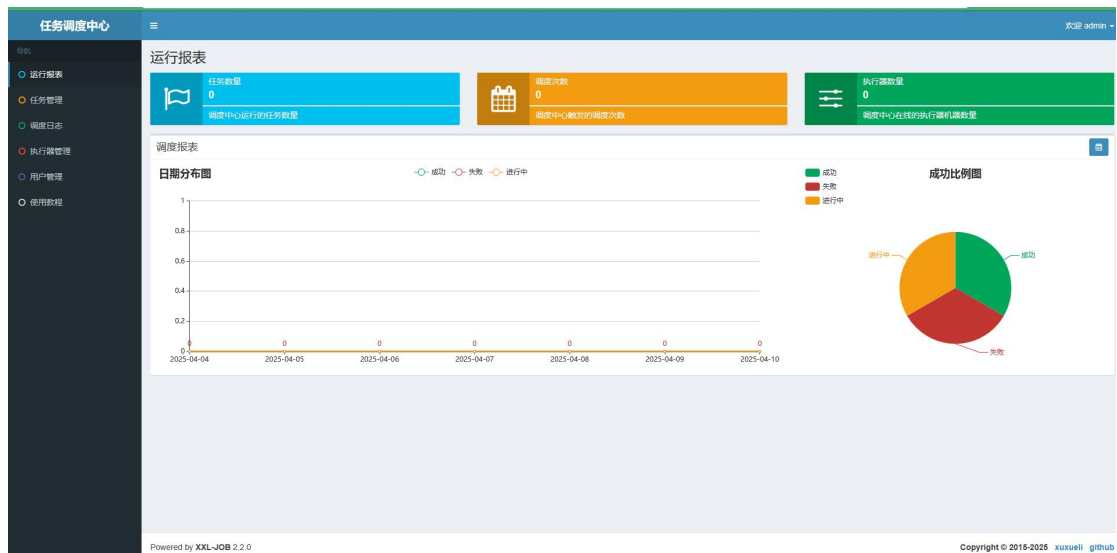
Plain Text
192.168.3.150 xxl-opensource.allthinker.com

```

访问地址：`http://xxl-opensource.allthinker.com/xxl-job-admin/toLogin`

账号：`admin`

密码：`MfY4aHJwTxihqqnoG`



5、数据库脚本执行

所有中间件部署完成后，在 `/data/mysql/k8s` 目录依次执行 `database.sql` 和其他脚本，`data.sql` 最后执行

七、AT-Work 项目完整部署

7.1 项目与命名空间创建

Rancher 新建项目 `at-work`，并创建以下命名空间：

- `atwork`：后端服务命名空间
- `atwork-front`：前端服务命名空间
- `im`：即时通讯服务命名空间
- `wiki`：维库服务命名空间

7.2 配置映射创建

7.2.1 cfg-server 配置映射

test

et-work

资源

应用商店

命名空间

成员

工具

仪表盘

编辑配置映射: cfg-server

名称

cfg-server

命名空间

atwork

配置映射

键 * 值

NACOS_DISCOVERY_URL

=

nacos.nacos.8848

-

SPRING_CLOUD_NACOS_CONFIG_NAMESPACE

=

282b0efc-75d8-488e-8c29-3037a06f64ed

-

SPRING_CLOUD_NACOS_DISCOVERY_NAMESPACE

=

282b0efc-75d8-488e-8c29-3037a06f64ed

-

SPRING_PROFILES_ACTIVE

=

prod

-

高级技巧：在键(key)输入栏中粘贴一行或多行的 key=value 键值对能够批量输入。

+ 添加配置映射值

从文件读取

标签/注释

用于描述资源的键值对。后跟重写配置注释: 键: nginx.ingress.kubernetes.io/rewrite-target; 值: /。

1 条规则已配置

保存

取消

- NACOS_DISCOVERY_URL: nacos.nacos.8848
- SPRING_CLOUD_NACOS_CONFIG_NAMESPACE: 282b0efc-75d8-488e-8c29-3037a06f64ed
- SPRING_CLOUD_NACOS_DISCOVERY_NAMESPACE: 282b0efc-75d8-488e-8c29-3037a06f64ed
- SPRING_PROFILES_ACTIVE: prod

7.2.2 cfg-ide 配置映射

编辑配置映射: cfg-ide

名称

cfg-ide

命名空间

group-1

配置映射

键 * 值

PATH

=

/home/coder/app/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

-

PROXY_DOMAIN

=

ide.allthinker.com

-

START_ARGS

=

--auth-url https://work.allthinker.com/https://ide.allthinker.cloud/login-ide/

-

TZ

=

Asia/Shanghai

-

高级技巧：在键(key)输入栏中粘贴一行或多行的 key=value 键值对能够批量输入。

+ 添加配置映射值

从文件读取

标签/注释

用于描述资源的键值对。后跟重写配置注释: 键: nginx.ingress.kubernetes.io/rewrite-target; 值: /。

1 条规则已配置

保存

取消

- PATH:
/home/coder/app/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
- TZ: Asia/Shanghai

7.2.3 cfg-idea 配置映射

编辑配置映射: cfg-idea

名称

cfg-idea

命名空间

group-1

配置映射

键 *

值

从文件读取

PATH_APP

=

/home/coder/app/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

—

START_ARGS

=

值

—

TZ

=

Asia/Shanghai

—

高级技巧: 在键Key输入框中粘贴一行或多行的 key=value 键值对能够批量输入。

+ 添加配置映射值

标签/注释

用于调度的键值对，后请重写配置注释 键 nginx.ingress.kubernetes.io/rewrite-target, 值 /。

1 条规则已配置

保存

取消

- PATH_APP:
/home/coder/app/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
- TZ: Asia/Shanghai

7.3 在 atwork 命名空间下部署后端服务

- 1、解压部署所需要的所有镜像，在目录/data/ide-share/images 集群里所有的机器中都解压 docker load -i ***.tar， 这目录下所有的镜像都需要解压

Plain Text
docker load -i ***.tar

升级服务

名称

user

工作负载类型

Deployment: 部署无状态应用 1 个 Pod

Docker 镜像 *

user-server:latest

命名空间

atwork

端口映射

+ 添加规则

环境变量

设置当前可见的环境变量，包括从其他资源加载环境变量注入的值。

环境变量

+ 添加变量

引用其他资源

类型 *

Config Map

源 *

cfg-server

键 *

All

前缀

+ 添加附加资源

2、按照 user 服务进行部署，其余所有后端服务都一样，没有特殊区别 部署完成如下：

命名空间: atwork					
☐	▶	Active	user	user-server:latest 1个 Pod / 创建时间: 2 months ago / 容器重启次数: 0	1
☐	▶	Active	task	task-server:latest 1个 Pod / 创建时间: 2 months ago / 容器重启次数: 0	1
☐	▶	Active	oauth	oauth-server:latest 1个 Pod / 创建时间: 2 months ago / 容器重启次数: 0	1
☐	▶	Active	monitor	monitor-server:latest 1个 Pod / 创建时间: 2 months ago / 容器重启次数: 0	1
☐	▶	Active	manage	manage-server:latest 1个 Pod / 创建时间: 2 months ago / 容器重启次数: 0	1
☐	▶	Active	gateway	gateway-server:latest 1个 Pod / 创建时间: 2 months ago / 容器重启次数: 0	1
☐	▶	Active	files	files-server:latest 1个 Pod / 创建时间: 2 months ago / 容器重启次数: 0	1
☐	▶	Active	disk	disk-server:latest 1个 Pod / 创建时间: 2 months ago / 容器重启次数: 0	1

7.4 AT Work 前端服务部署

7.4.1 task 前端部署

修改 `/data/ide-share/task-nginx` 目录下 `config.json` 和 `nginx.conf` 中的 IP 地址，配置对外端口及数据卷，完成部署；端口注意多写一个，对外使用。

名称task

工作负载类型Deployment: 部署无状态应用 1 个 Pod

Docker 镜像 *task-web:latest

命名空间atwork-front

端口映射

端口名称	容器端口 *	协议	网络模式	主机端口映射 *
80tcp02	80	TCP	集群 IP (集群内部访问)	与容器端口相同
tcp8089	80	TCP	HostPort (仅 Pod 所在主机端口可访问)	8089

+ 添加规则

环境变量

设置容器可见的环境变量，包括从其他资源(如文本等)注入的值。

键 *	值
NGINX_MAX_BODY_SIZE	50m
NGINX_PORT	80
NGINX_PROXY_PASS	http://gateway.atwork:8765

+ 添加变量

数据卷部分

数据卷

持久化和共享独立于单个容器生命周期的数据

卷名

卷类型

删除卷

config

主机映射卷

主机路径

主机路径类型检查

/data/ide-share/task-nginx/config.json

不检查目标路径

容器路径

子路径(仅适用于数据卷挂载, 不适用于主机路径映射)

只读

/usr/share/nginx/html/config.json

☐

添加映射

卷名

卷类型

删除卷

nginx

主机映射卷

主机路径

主机路径类型检查

/data/ide-share/task-nginx/nginx.conf

文件: 不存在则创建

容器路径

子路径(仅适用于数据卷挂载, 不适用于主机路径映射)

只读

/etc/nginx/conf.d/nginx.conf

☐

添加映射

7.4.2 disk 前端部署

修改/data/ide-share/disk-nginx 目录下配置文件 IP 地址，配置数据卷后部署

名称

工作负载类型

disk

Deployment 部署无状态应用 1 个 Pod

Docker 镜像

命名空间

disk-web:latest

atwork-front

端口映射

端口名称

容器端口

协议

网络模式

主机端口监听

80tcp02

80

TCP

集群 IP (集群内部访问)

与容器端口相同

tcp8090

80

TCP

HostPort (仅 Pod 所在主机端口可访问)

8099

添加规则

环境变量

全部展开

设置容器可见的环境变量, 包括从其他资源如配置文件注入的值。

环境变量

键

值

NGINX_PORT

=

80

NGINX_PROXY_PASS

=

http://gateway.atwork:8765

NGINX_ROOT_HOST

=

allthinker.com

添加变量

数据卷部分

数据卷

持久化和共享独立于单个容器生命周期的数据

卷名

disk-front

卷类型

主机映射卷

删除卷

主机路径

/mnt-at

主机路径类型检查

目录: 不存在则创建

容器路径

/mnt-at

子路径(仅适用于数据卷挂载, 不适用于主机路径映射)

只读

☐

添加映射

卷名

config

卷类型

主机映射卷

删除卷

主机路径

/data/ide-share/disk-nginx/config.json

主机路径类型检查

文件: 不存在则创建

容器路径

/usr/share/nginx/html/config.json

子路径(仅适用于数据卷挂载, 不适用于主机路径映射)

只读

☐

添加映射

卷名

nginx

卷类型

主机映射卷

删除卷

主机路径

/data/ide-share/disk-nginx/nginx.conf

主机路径类型检查

文件: 不存在则创建

容器路径

/etc/nginx/nginx.conf

子路径(仅适用于数据卷挂载, 不适用于主机路径映射)

只读

☐

添加映射

7.5 即时通讯服务部署

1、MinIO 组件部署（其余组件类似，解压之后运行就行，注意修改 dockercompose.yml 里面的 ip 地址）

Plain Text

```
cd /data/aoao/software/minio
docker load -i minio.tar
docker compose up -d
```

2、前端部署

Plain Text

```
cd /data/aoao/web
docker load -i im-web.tar
docker compose up -d
```

nginx.conf 配置修改成虚拟机 ip 或者服务器 ip

3、后端部署

平台服务：

```
Plain Text
cd /data/aoao/server/platform
# 修改.env 文件数据库、Redis 等 IP 配置
docker load -i at-platform.tar
docker compose up -d
```

主服务：

```
Plain Text
cd /data/aoao/server/server
# 修改.env 文件配置
docker load -i at-server.tar
docker compose up -d
```

7.6 维库服务部署

```
Plain Text
cd /data/wiki
# 解压所有镜像文件
docker load -i *.tar
# 修改 dockercompose.yml 内 IP 地址
docker compose up -d
```

八、全局部署注意事项

8.1 数据库基础数据

所有服务部署完成后，务必导入完整数据库基础数据，保证服务初始化正常

8.2 K3s 集群 Token 获取

K3s requestToken 需要在 k3s 主节点获取：

```
Plain Text
kubectl -n kube-system describe $(kubectl -n kube-system get
secret -n kube-system -o name | grep namespace) | grep token
```

[illegible]

8.3 主节点 NFS 服务端配置文件的位置和内容

配置路径: `/etc/exports`

完整配置内容：

Plain Text

```
/data/nfs-share *(insecure,rw,sync,no_root_squash)
/data/redis *(insecure,rw,sync,no_root_squash)
/data/mysql/k8s *(insecure,rw,sync,no_root_squash)
/data/documentserver *(insecure,rw,sync,no_root_squash)
/data/ide-share *(insecure,rw,sync,no_root_squash)
/mnt-at *(insecure,rw,sync,no_root_squash)
```

8.4 定时任务配置规范

1、添加相关的数据库的数据 图示如下

AppName	请输入AppName	名称	名称	搜索	新增
每页	10	↓	条记录		
AppName	名称	注册方式	OnLine 机器地址	操作	
at-disk-job-executor	云空间定时器	自动注册	查看 (1)	操作	↓
at-manage-job-executor	服务管理定时器	自动注册	查看 (1)	操作	↓
at-task-job-executor	任务管理定时器	自动注册	无	操作	↓

3、相关任务

执行器

服务管理定时器

全部

请输入任务描述

请输入JobHandler

请输入负责人

搜索

新增

每页

10

↓

象记录

任务ID	任务描述	运行模式	Cron	负责人	状态	操作
29	定时采集用户使用指标参数	BEAN: userActionTaskSyncUsers	0 0 1 * * ?	admin	RUNNING	操作
28	容器磁盘监控	BEAN: diskMonitorJobHandler	0 0 * * * ?	admin	RUNNING	操作
27	处理 VPS 待操作记录	BEAN: processVpsOptRecords	1/3 * * * * ?	admin	RUNNING	操作
26	团队权益过期提醒及过期处理	BEAN: sendTeamAdminOffLegalRightExpire	0 0 2 * * ?	admin	RUNNING	操作
25	清空过期权益磁盘资源	BEAN: clearBoxAndCoding	0 0 1 * * ?	admin	RUNNING	操作
24	定时计算ide的使用时长	BEAN: FixedTimeSettlementHandler	0 0 30 * * * ?	admin	RUNNING	操作
23	定时计算ide的使用时长	BEAN: FixedTimeSettlementHandler	0 0 30 * * * ?	admin	RUNNING	操作

第 1 页 (总共 1 页, 7 条记录)

上页

1

4、nacos 上的配置需要与执行器的 AppName 要保证关联性，配置图示如下

```

xxl:
  job:
    enable: true
    ### 执行器通讯TOKEN [选填]: 非空时启用;
    accessToken: ''
    admin:
      # 调度中心部署地址 [选填]: 如调度中心集群部署存在多个地址则用逗号分隔。执行器将会使用该地址进行“执行器心跳注册”和“任务结果回调”; 为空则关闭自动注册;
      # (每个环境需要单独配置)
      addresses: http://xxl-job-job:8888/xxl-job-admin
    executor:
      ### 执行器注册 [选填]: 优先使用该配置作为注册地址, 为空时使用内嵌服务 “IP:PORT” 作为注册地址。从而更灵活的支持容器类型执行器动态IP和动态映射端口问题。
      address: ''
      ### 执行器IP [选填]: 默认为空表示自动获取IP, 多网卡时可手动设置指定IP, 该IP不会绑定Host仅作为通讯实用; 地址信息用于 “执行器注册” 和 “调度中心请求并分发任务”;
      ip: ''
      ### 执行器端口号 [选填]: 小于等于0则自动获取; 默认端口为9999, 单机部署多个执行器时, 注意要配置不同执行器端口号;
      port: 25(server.port)
      ### 执行器AppName [选填]: 执行器心跳注册分组依据; 为空则关闭自动注册
      appname: at-${microservice.service-simple-name}-job-executor
      ### 执行器运行日志文件存储磁盘路径 [选填]: 需要对该路径拥有读写权限; 为空则使用默认路径;
      logpath: /opt/logs/jobs
      ### 执行器日志文件保存天数 [选填]: 过期日志自动清理, 限制值大于等于3时生效; 否则, 如-1, 关闭自动清理功能;
      logretentiondays: 3

```

8.5 K8s 配置一致性要求

这个文件的内容与 /etc/rancher/k3s/k3s.yaml 的文件内容保持一致

bak	文件夹	2025/04/14 18:35	drwxr-xr-x	root/root
box-deployment.yml	2.8 KB Yaml 源文...	2025/04/14 18:35	-rw-r--r--	root/root
box-pod.yml	2.6 KB Yaml 源文...	2025/04/14 18:35	-rw-r--r--	root/root
box-service.yml	128 B Yaml 源文...	2025/04/14 18:35	-rw-r--r--	root/root
cloud-deployment.y...	2.9 KB Yaml 源文...	2025/04/14 18:35	-rw-r--r--	root/root
cloud-deployment-...	2.1 KB Yaml 源文...	2025/04/14 18:35	-rw-r--r--	root/root
ide-deployment.yml	2.9 KB Yaml 源文...	2025/04/14 18:35	-rw-r--r--	root/root
ide-deployment-1.y...	2.1 KB Yaml 源文...	2025/04/14 18:35	-rw-r--r--	root/root
ide-pod.yml	2.3 KB Yaml 源文...	2025/04/14 18:35	-rw-r--r--	root/root
ide-pod-1.yml	1.8 KB Yaml 源文...	2025/04/14 18:35	-rw-r--r--	root/root
ide-service.yml	101 B Yaml 源文...	2025/04/14 18:35	-rw-r--r--	root/root
ide-service-1.yml	101 B Yaml 源文...	2025/04/14 18:35	-rw-r--r--	root/root
k8s_config_box_at6...	2.9 KB Yaml 源文...	2025/04/14 18:35	-rw-r--r--	root/root
k8s_config_box.yml	2.9 KB Yaml 源文...	2025/04/14 18:35	-rw-r--r--	root/root
k8s_config_ide.yml	2.9 KB Yaml 源文...	2025/04/14 18:35	-rw-r--r--	root/root
k8s_config_prod.yml	2.9 KB Yaml 源文...	2025/04/15 13:27	-rw-r--r--	root/root
k8s_config.yml	2.9 KB Yaml 源文...	2025/04/15 13:23	-rw-r--r--	root/root

k8s api 服务访问 token, 获取方式如下:

```
kubectl -n kube-system describe $(kubectl -n kube-system get secret -n kube-system -o name | grep namespace) | grep token
```

8.6 RabbitMQ 用户初始化

进入 RabbitMQ 容器执行以下命令创建管理员账号:

```

Plain Text
rabbitmqctl add_user admin Gg88Ef983MFi3AfGtmXep6gk
rabbitmqctl set_user_tags admin administrator
rabbitmqctl set_permissions -p / admin ".*" ".*" ".*"
rabbitmqctl list_users

```

```
Listing users ...  
user      tags  
management [management]  
mall      [administrator]  
admin     [administrator]  
bash-5.0$
```